

Web Components I

Resumen del fastbook · Autor original: Miguel Ángel Muñoz

Este documento resume las seis lecciones del fastbook sobre Web Components: qué son, custom elements, plantillas HTML, Shadow DOM y su relación con frameworks como React, Angular, Vue o Polymer.

1. Qué son los web components

Son elementos personalizados (por ejemplo `<my-component>`) que se importan y se usan como cualquier otra etiqueta HTML nativa. Igual que `<button>` o `<form>` traen estilos predefinidos, los web components pueden llevar los suyos propios.

- **Encapsulan** estilos y funcionalidad, sin afectar ni verse afectados por el entorno.
- Permiten **exponer formas de interactuar** (estilos personalizados, HTML interno) para adaptarlos al contexto.
- Se pueden **importar** en archivos HTML sin copiar y pegar código.

2. Custom elements

Son un conjunto de APIs de JavaScript que permiten crear elementos personalizados y definir su comportamiento. Se apoyan en las **APIs web** (funcionalidades del navegador ya desarrolladas y listas para usar), como FetchAPI, APIs de audio/vídeo, Web Storage API o la API de geolocalización.

Objeto CustomElementRegistry

- `define()`: define un nuevo elemento personalizado.
- `get()`: devuelve el constructor de un elemento.
- `getName()`: devuelve el nombre a partir del constructor.
- `upgrade()`: actualiza un elemento personalizado.
- `whenDefined()`: promesa que se resuelve cuando el elemento se define.

Ejemplo básico: `window.customElements.define('my-button', class MyButton extends HTMLElement { ... })`. La clase debe extender de **HTMLElement**, definir un constructor `()` y llamar a `super()`. Dentro, `this` referencia siempre a la instancia concreta, por lo que cada componente tiene su propio scope.

Ciclo de vida (callbacks)

- **connectedCallback**: al añadirse al DOM por primera vez.
- **disconnectedCallback**: al quitarse del DOM.
- **adoptedCallback**: al moverse a otro documento.

- **attributeChangedCallback**: al modificarse un atributo observado.

3. Plantillas HTML

Permiten definir estructuras HTML reutilizables que **no se renderizan por sí mismas**. Se apoyan en dos etiquetas:

- `<template>`: contiene la estructura a reutilizar; se referencia con `document.getElementById()` y se clona con `template.content.cloneNode(true)`.
- `<slot>`: define zonas personalizables desde fuera del componente (atributo `name` y contenido por defecto opcional). El contenido se sustituye añadiendo el atributo `slot` a la etiqueta insertada.

4. Shadow DOM

Es una versión “en la sombra” del DOM: un DOM oculto y encapsulado, enlazado a un nodo concreto. El DOM (document object model) es la estructura jerárquica del documento HTML accesible vía `document`.

- Los estilos globales **no afectan** al contenido dentro del Shadow DOM, y viceversa.
- Se crea con `attachShadow({ mode: 'open' })`; se puede tener tantos Shadow DOM como se quiera, siempre ligados a un elemento (shadow host).
- Puede ser **open** (visible desde fuera) o **closed** (oculto).
- No es exclusivo de los web components: puede usarse para encapsular cualquier parte del HTML.

Se diferencia del **Virtual DOM** (usado por React, Vue, etc.), que es una copia en memoria del DOM para optimizar renderizados agrupando cambios, sin relación con el Shadow DOM.

5. React, Angular, Vue, Polymer

Estos frameworks no inventan algo distinto: construyen sus propios web components sobre las mismas tecnologías vistas (custom elements, Shadow DOM, templates), añadiendo una capa de simplificación y funcionalidades por defecto.

Mensaje clave del autor: aprender primero los fundamentos de JavaScript y de los web components (antes que un framework concreto) permite entender el “por qué” de las cosas y da más capacidad de evolucionar como desarrollador, en vez de depender únicamente de un framework.

6. Conclusión

El fastbook recomienda practicar mucho con el Shadow DOM, crear componentes propios, pasarles parámetros y preparar estilos internos para consolidar todos los conceptos vistos.